

Python - Loop Lists

Loop Through List Items

Looping through list items in Python refers to iterating over each element within a list. We do so to perform the desired operations on each item. These operations include list modification, conditional operations, string manipulation, data analysis, etc.

Python provides various methods for looping through **list** items, with the most common being the **for loop**. We can also use the **while loop** to iterate through list items, although it requires additional handling of the loop control variable explicitly i.e. an index.

Loop Through List Items with For Loop

A for loop in Python is used to iterate over a sequence (like a list, tuple, dictionary, string, or range) or any other iterable object. It allows you to execute a block of code repeatedly for each item in the sequence.

In a for loop, you can access each item in a sequence using a variable, allowing you to perform operations or logic based on that item's value. We can loop through list items using for loop by iterating over each item in the list.

Syntax

Following is the basic syntax to loop through items in a list using a for loop in Python –

```
for item in list:  
    # Code block to execute
```

Example

In the following example, we are using a for loop to iterate through each element in the list "lst" and retrieving each element followed by a space on the same line –

```
lst = [25, 12, 10, -21, 10, 100]
for num in lst:
    print (num, end = ' ')
```

Output

Following is the output of the above code –

```
25 12 10 -21 10 100
```

Loop Through List Items with While Loop

A while loop in Python is used to repeatedly execute a block of code as long as a specified condition evaluates to "True".

We can loop through list items using while loop by initializing an index variable, then iterating through the list using the index variable and incrementing it until reaching the end of the list.

An index variable is used within a loop to keep track of the current position or index in a sequence, such as a list or array. It is generally initialized before the loop and updated within the loop to iterate over the sequence.

Syntax

Following is the basic syntax for looping through items in a list using a while loop in Python –

```
while condition:
    # Code block to execute
```

Example

In the below example, we iterate through each item in the list "my_list" using a while loop. We use an index variable "index" to access each item sequentially, incrementing it after each iteration to move to the next item –

```
my_list = [1, 2, 3, 4, 5]
index = 0

while index < len(my_list):
    print(my_list[index])
    index += 1
```

Output

Output of the above code is as follows –

```
1
2
3
4
5
```

Loop Through List Items with Index

An index is a numeric value representing the position of an element within a sequence, such as a list, starting from 0 for the first element.

We can loop through list items using index by iterating over a range of indices corresponding to the length of the list and accessing each element using the index within the loop.

Example

This example initializes a list "lst" with integers and creates a range of indices corresponding to the length of the list. Then, it iterates over each index in the range and prints the value at that index in the list "lst" –



```
lst = [25, 12, 10, -21, 10, 100]
indices = range(len(lst))
for i in indices:
    print ("lst[{}]: ".format(i), lst[i])
```

Output

We get the output as shown below –

```
lst[0]: 25
lst[1]: 12
lst[2]: 10
lst[3]: -21
lst[4]: 10
lst[5]: 100
```

Iterate using List Comprehension

A list comprehension in Python is a concise way to create lists by applying an expression to each element of an iterable. These expressions can be arithmetic operations, function calls, conditional expressions etc.

We can iterate using list comprehension by specifying the expression and the iterable (like a list, tuple, dictionary, string, or range). Following is the syntax –

```
[expression for item in iterable]
```

This applies the expression to each item in the iterable and creates a list of results.

Example

In this example, we use list comprehension to iterate through each number in a list of numbers, square each one, and store the squared result in the new list "squared_numbers" –

```
numbers = [1, 2, 3, 4, 5]
squared_numbers = [num ** 2 for num in numbers]
print (squared_numbers)
```

Output

We get the output as shown below –

```
[1, 4, 9, 16, 25]
```

Iterate using the enumerate() Function

The enumerate() function in Python is used to iterate over an iterable object while also providing the index of each element.

We can iterate using the enumerate() function by applying it to the iterable. Following is the syntax –

```
for index, item in enumerate(iterable):
```

This provides both the index and item of each element in the iterable during iteration

Example

In the following example, we are using the enumerate() function to iterate through a list "fruits" and retrieve each fruit along with its corresponding index –



```
fruits = ["apple", "banana", "cherry"]  
for index, fruit in enumerate(fruits):  
    print(index, fruit)
```

Output

We get the output as shown below –

```
0 apple  
1 banana  
2 cherry
```

Python Tutorial
Java Tutorial
C++ Tutorial
C Programming Tutorial
C# Tutorial
PHP Tutorial
R Tutorial
HTML Tutorial
CSS Tutorial
JavaScript Tutorial
SQL Tutorial

TRENDING TECHNOLOGIES

Cloud Computing Tutorial
Amazon Web Services Tutorial
Microsoft Azure Tutorial
Git Tutorial
Ethical Hacking Tutorial
Docker Tutorial
Kubernetes Tutorial
DSA Tutorial
Spring Boot Tutorial
SDLC Tutorial
Unix Tutorial

CERTIFICATIONS

Business Analytics Certification
Java & Spring Boot Advanced Certification
Data Science Advanced Certification
Cloud Computing And DevOps
Advanced Certification In Business Analytics
Artificial Intelligence And Machine Learning
DevOps Certification
Game Development Certification
Front-End Developer Certification



COMPILERS & EDITORS

[Online Java Compiler](#)

[Online Python Compiler](#)

[Online Go Compiler](#)

[Online C Compiler](#)

[Online C++ Compiler](#)

[Online C# Compiler](#)

[Online PHP Compiler](#)

[Online MATLAB Compiler](#)

[Online Bash Terminal](#)

[Online SQL Compiler](#)

[Online Html Editor](#)

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |

[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2026. All Rights Reserved.